

BT5110: Tutorial 1 — Relational Model

Pratik Karmakar

School of Computing,
National University of Singapore

AY26/27 S1



What is the relational model? (in plain words)

- Think **spreadsheets**: each **table** is a sheet, **rows** are items, **columns** are properties.
- Formally, a table is a **relation**; a row is a **tuple**; a column is an **attribute**.
- Each table has a rule for what counts as a valid row: that rulebook is the **schema**.
- Tables can be **linked**—a value in one table points to a matching value in another.
- Why it's powerful: simple structure, **strong guarantees** (constraints, transactions), and **declarative** querying (SQL).

Everyday analogy

Students table

- One row per student
- Columns: email, name, faculty, department, joined
- **Primary key** (email) uniquely identifies a student

Books table

- One row per book title
- Columns: isbn13 (PK), isbn10, title, authors, publisher, year

Copies & Loans

- `copy(owner, book, copy, ...)`
- `loan(owner, book, copy, borrowed, returned)`
- **Foreign keys** connect:
 - `copy.owner` → `student.email`
 - `copy.book` → `book.isbn13`
 - `loan.(owner, book, copy)` → `copy`

Keys, relationships, and constraints

- **Primary key (PK):** a column (or set) that uniquely identifies each row.
- **Foreign key (FK):** a column whose values must match a PK in another table.
- **Domain & CHECK constraints:** control valid values (e.g., dates, non-negative pages).
- **Entity integrity:** PK cannot be NULL.
- **Referential integrity:** FK must reference an existing PK (or be NULL if allowed).
- These rules keep data **consistent** across tables.

How we ask questions (SQL mirrors simple ops)

Core ideas

- **Filter** rows (*selection*, σ):
WHERE
- **Pick** columns (*projection*, π):
SELECT list
- **Match** tables (*join*, $\bowtie$): JOIN
... ON
- **Summarize** (*group/aggregate*):
GROUP BY, COUNT, SUM, ...

Example questions

- “Which copies are currently on loan?”
- “Which students own at least two books?”
- “How many Chemistry students borrowed in 2024?”

Takeaway: you describe *what* you want; the database figures out *how* to get it efficiently (query optimization).

Why Relational Algebra?

- **Foundation of SQL:** Relational algebra provides the formal, mathematical basis for relational databases.
- Defines **operations on relations** that always produce relations.
- Ensures queries are **precise**, **composable**, and **optimizable**.
- SQL = (mostly) declarative syntax built on these algebraic ideas.

Core Operators

- **Selection (σ)**: Pick rows that satisfy a condition.
 $\sigma_{\text{year}=2024}(\text{Student})$
- **Projection (π)**: Pick certain columns. $\pi_{\text{name, email}}(\text{Student})$
- **Renaming (ρ)**: Rename relation/attributes. $\rho_S(\text{Student})$
- **Union ($\cup$), Difference ($-$), Intersection ($\cap$)**: Set-like ops (schemas must match).
- **Cartesian Product ($\times$)**: Pair all tuples across two relations.
- **Join ($\bowtie$)**: Combine rows across relations on matching attributes.

Examples and SQL Mapping

Relational Algebra

- $\pi_{\text{name}}(\sigma_{\text{faculty}=\text{'SoC'}}(\text{Student}))$

```
1 SELECT name
2 FROM student
3 WHERE faculty = 'SoC';
```

- $\text{Student} \bowtie_{\text{student.email}=\text{copy.owner}} \text{Copy}$

```
1 SELECT *
2 FROM student s
3 JOIN copy c
4   ON s.email = c.owner;
```

- $\pi_{\text{isbn13}}(\text{Book}) - \pi_{\text{book}}(\text{Copy})$

```
1 SELECT isbn13
2 FROM book
3 EXCEPT
4 SELECT book
5 FROM copy;
```


Takeaway

- Relational algebra = the **mathematical core** of querying.
- SQL extends it with ordering, grouping, aggregation, etc.
- Understanding algebra clarifies how SQL queries work “under the hood.”

Case Description

Students at the National University of Ngendipura (NUN) buy, lend, and borrow books. Your company, Apasaja Private Limited, is commissioned by NUN Students Association (NUNStA) to implement an online book exchange system that records information about students and the books they own, borrow, and lend.

The database stores:

- **Students:** name, faculty, department, joined date, graduation date. Identifier: email.
- **Books:** title, format, pages, language, authors, publisher, publication date, ISBN-10, ISBN-13.
- **Loans:** per-copy borrow and return dates.

A book may exist with no current owner (owner graduated, or a lecturer recommended it before any student bought it). Auditing: books, copies, and students are retained while referenced by an existing copy or loan record.

Q1. Data Definition Language (DDL)

(a) Download from Canvas ▷ Files ▷ Cases ▷ Book

Exchange:

NUNStASchema.sql, NUNStAClean.sql, NUNStAStudent.sql,
NUNStABook.sql, NUNStACopy.sql, NUNStALoan.sql.

What they do:

- **Clean Up** (NUNStAClean.sql): drops existing tables —
DROP TABLE IF EXISTS.
- **Schema** (NUNStASchema.sql): defines the four relations and
their constraints — CREATE TABLE, PRIMARY KEY, FOREIGN
KEY, UNIQUE, NOT NULL, CHECK.
- **Data** (NUNStAStudent/Book/Copy/Loan.sql): populate
the tables — INSERT INTO student/book/copy/loan
(order matters).

Q1. Finding and Fixing the Error

(b) Running NUNStASchema.sql as downloaded raises an error:

```
1 ERROR:  relation "copy" does not exist
```

Cause: the file creates tables in the order book, student, loan, copy — but loan declares

```
1 FOREIGN KEY (owner, book, copy) REFERENCES copy(owner, book, copy
  )
```

so CREATE TABLE loan fails because copy does not exist yet.

Fix: create tables in dependency order — book/student → copy → loan — then populate student/book → copy → loan.

Cleanup is the reverse.

Creating the Database

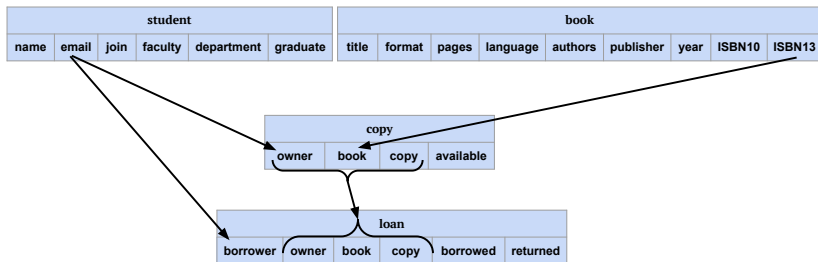


Figure 1: student and book are independent tables (relations). copy relies on student for its owner attribute and on book for its book attribute. loan relies on copy for (owner, book, copy) attributes and on student for its borrower attribute.

Creating the Database

From Figure 1 it is clear that we need to populate either the student table or the book table first. Then we should populate copy and loan should be the last.

¹The order of `NUNStASStudent.sql` and `NUNStABook.sql` are interchangeable.

²You need to be in the same directory of these files for these to run. Otherwise provide the entire file paths after `\i.`

Creating the Database

From Figure 1 it is clear that we need to populate either the student table or the book table first. Then we should populate copy and loan should be the last.

How would you find the order when the number of tables (relations) is much higher and have complex set of dependencies?

¹The order of `NUNStASStudent.sql` and `NUNStABook.sql` are interchangeable.

²You need to be in the same directory of these files for these to run. Otherwise provide the entire file paths after `\i.`

Creating the Database

From Figure 1 it is clear that we need to populate either the student table or the book table first. Then we should populate copy and loan should be the last.

How would you find the order when the number of tables (relations) is much higher and have complex set of dependencies? Use Topological Sorting

¹The order of `NUNStASudent.sql` and `NUNStABook.sql` are interchangeable.

²You need to be in the same directory of these files for these to run. Otherwise provide the entire file paths after `\i.`

Creating the Database

From Figure 1 it is clear that we need to populate either the student table or the book table first. Then we should populate copy and loan should be the last.

How would you find the order when the number of tables (relations) is much higher and have complex set of dependencies? Use Topological Sorting

In psql run¹:

```
1      \i NUNStAStudent.sql;  
2      \i NUNStABook.sql;  
3      \i NUNStACopy.sql;  
4      \i NUNStALoan.sql;  
5
```

Your database is now ready².

¹The order of NUNStAStudent.sql and NUNStABook.sql are interchangeable.

²You need to be in the same directory of these files for these to run. Otherwise provide the entire file paths after \i.

Q1. Primary Keys

(c) Primary key of each relation:

- book: ISBN13
- student: email
- copy: (owner, book, copy)
- loan: (borrowed, borrower, owner, book, copy)

Q1. Foreign Keys

(d) Referencing $\rightarrow$ referenced relation/attribute(s):

- `copy.owner` $\rightarrow$ `student.email`
- `copy.book` $\rightarrow$ `book.ISBN13`
- `loan.borrower` $\rightarrow$ `student.email`
- `loan.(owner, book, copy)` $\rightarrow$ `copy.(owner, book, copy)`

Q1. Composite Primary Keys

(e) Which relations need more than one attribute to be uniquely identified?

- **copy** — (**owner, book, copy**): an owner can own different books, and more than one copy of the *same* book; the copy number distinguishes those copies.
- **loan** — (**borrowed, borrower, owner, book, copy**): uniquely identifies a loan *event*, i.e. a particular borrower borrowing a particular physical copy on a particular date.

Q1. Constraints — UNIQUE, NOT NULL, CHECK

(f) Examples and the business rule each enforces:

- **UNIQUE** `book.ISBN10` — no two book records may share the same ISBN-10.
- **NOT NULL** `student.name`, `joined`, `faculty`, `department` — required student information cannot be missing.
- **CHECK** `book.format = 'paperback' OR 'hardcover'` — only permitted book formats are stored.
- **CHECK** `student.graduate >= joined` — a graduation date cannot be before the student joined.
- **CHECK** `copy.copy > 0` — copy numbers must be positive.
- **CHECK** `loan.returned >= borrowed` — a return date cannot be before the borrowing date.

Q2. Insert / Delete / Update — Book Inserts

(a) Insert a new book:

```
1  INSERT INTO book VALUES (  
2    'An Introduction to Database Systems',  
3    'paperback',  
4    640,  
5    'English',  
6    'C. J. Date',  
7    'Pearson',  
8    '2003-01-01',  
9    '0321197844',  
10   '978-0321197849'  
11 );  
12 -- Verify:  
13 SELECT * FROM book;
```

Q2. Book Variants & Keys

(b) Same book, different ISBN13 (unique isbn10 violated):

```
1  INSERT INTO book VALUES (  
2    'An Introduction to Database Systems',  
3    'paperback',  
4    640,  
5    'English',  
6    'C. J. Date',  
7    'Pearson',  
8    '2003-01-01',  
9    '0321197844',  
10   '978-0201385908'  
11 );
```

Q2. Book Variants & Keys

(c) Same book, different ISBN10 (PK isbn13 violated):

```
1  INSERT INTO book VALUES (  
2    'An Introduction to Database Systems',  
3    'paperback',  
4    640,  
5    'English',  
6    'C. J. Date',  
7    'Pearson',  
8    '2003-01-01',  
9    '0201385902',  
10   '978-0321197849'  
11 );
```


Q2. Insert Students

(d) Insert a new student by explicitly naming the target columns.
What value is stored in any omitted column?

```
1  INSERT INTO student (email, name, joined, faculty, department)
   VALUES (
2    'ben.lim@u.nun.edu',
3    'BEN LIM',
4    '2026-08-10',
5    'School of Computing',
6    'CS'
7  );
```

Insertion succeeds; the omitted graduate column is NULL.

Q2. Update & Delete

(e) Change department name from 'CS' to 'Computer Science':

```
1 UPDATE student
2 SET department = 'Computer Science'
3 WHERE department = 'CS';
```

Every 'CS' row is updated: 24 students originally, plus Ben Lim from (d) $\Rightarrow$ **25 rows**.

(f) Case-sensitive delete:

```
1 DELETE FROM student WHERE department = 'chemistry';
```

0 rows match — PostgreSQL string comparison is case-sensitive; no row stores lowercase 'chemistry'.

Q2. Update & Delete

(g)

```
1 DELETE FROM student WHERE department = 'Chemistry';
```

3 rows match, but the DELETE **fails**: those students are still referenced by foreign keys in copy and/or loan, so referential integrity blocks the deletion — none of the 3 rows is removed.

Comparing with (f): (f) matches 0 rows (case mismatch); (g) matches 3 rows but referential integrity prevents the deletion.

Q3. Modifying the Schema — Drop Redundant Availability

(a) Availability is derivable: a copy is unavailable iff it has an open loan.

```
1  -- Unreturned loans
2  SELECT owner, book, copy, returned
3  FROM loan
4  WHERE returned ISNULL;
5
6  -- Remove redundant column
7  ALTER TABLE copy
8  DROP COLUMN available;
```

Q3. Modifying the Schema — Drop Redundant Availability

```
1  -- View that recomputes availability
2  CREATE OR REPLACE VIEW copy_view (owner, book, copy, available)
   AS (
3    SELECT DISTINCT c.owner, c.book, c.copy,
4    CASE
5      WHEN EXISTS (
6        SELECT * FROM loan l
7        WHERE l.owner = c.owner
8              AND l.book = c.book
9              AND l.copy = c.copy
10             AND l.returned ISNULL
11      ) THEN 'FALSE'
12      ELSE 'TRUE'
13    END
14    FROM copy c
15  );
```

Q3. Modifying the Schema — Drop Redundant Availability

```
1  -- Example update attempt (requires INSTEAD OF trigger/rule in
   practice)
2  UPDATE copy_view
3  SET owner = 'tikki@google.com'
4  WHERE owner = 'tikki@gmail.com';
5
6  -- Drop when done
7  DROP VIEW copy_view;
```

Q3. Normalize Department → Faculty Mapping

(b) department → faculty (FD) — store mapping once.

```
1 CREATE TABLE department (  
2     department VARCHAR(32) PRIMARY KEY,  
3     faculty     VARCHAR(62) NOT NULL  
4 );  
5  
6 INSERT INTO department  
7 SELECT DISTINCT department, faculty  
8 FROM student;  
9  
10 ALTER TABLE student  
11 DROP COLUMN faculty;  
12  
13 ALTER TABLE student  
14 ADD FOREIGN KEY (department) REFERENCES department(department);
```

Questions?

Drop a mail at: pratik.karmakar@u.nus.edu